Beyond Linear LLM Invocation: An Efficient and Effective Semantic Filter Paradigm

Nan Hou
The Chinese University of Hong Kong
nhou@se.cuhk.edu.hk

Kangfei Zhao
Beijing Institute of Technology
zkf1105@gmail.com

Jiadong Xie
The Chinese University of Hong Kong
jdxie@se.cuhk.edu.hk

Jeffrey Xu Yu
HKUST (Guangzhou)
jeffreyxuyu@hkust-gz.edu.cn

ABSTRACT

Large language models (LLMs) are increasingly used for semantic query processing over large corpora. A set of semantic operators derived from relational algebra has been proposed to provide a unified interface for expressing such queries, among which the semantic filter operator serves as a cornerstone. Given a table T with a natural language predicate e, for each tuple in the relation, the execution of a semantic filter proceeds by constructing an input prompt that combines the predicate e with its content, querying the LLM, and obtaining the binary decision. However, this tuple-by-tuple evaluation necessitates a complete linear scan of the table, incurring prohibitive latency and token costs. Although recent work has attempted to optimize semantic filtering, it still does not break the linear LLM invocation barriers. To address this, we propose Clustering-Sampling-Voting (CSV), a new framework that reduces LLM invocations to sublinear complexity while providing error guarantees. CSV embeds tuples into semantic clusters, samples a small subset for LLM evaluation, and infers cluster-level labels via two proposed voting strategies: UniVote, which aggregates labels uniformly, and SimVote, which weights votes by semantic similarity. Moreover, CSV triggers re-clustering on ambiguous clusters to ensure robustness across diverse datasets. The results conducted on real-world datasets demonstrate that CSV reduces the number of LLM calls by 1.28-355× compared to the state-of-the-art approaches, while maintaining comparable effectiveness in terms of Accuracy and F1 score.

PVLDB Reference Format:
Nan Hou, Kangfei Zhao, Jiadong Xie, and Jeffrey Xu Yu. Beyond Linear LLM Invocation: An Efficient and Effective Semantic Filter Paradigm. PVLDB, 20(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/Anto-an/CSV_SemanticFilter.

1 INTRODUCTION

Large language models (LLMs) have rapidly become essential tools in modern data processing, demonstrating an unprecedented capacity to understand and reason over natural language queries using their embedded world knowledge. A wide range of complex analytical applications have been empowered by LLMs, including multi-document summarization [51], causal relation extraction [10], and legal contract analysis [9]. As semi-structured and unstructured data continue to grow, there is an increasing need for analytical systems to support bulk semantic query processing, i.e., evaluating a large volume of data through LLM-based semantic transformations. Unlike classical relational operators that follow symbolic reasoning, semantic query processing is oriented towards pragmatics and relies on context, shared knowledge, and consensus for reasoning.

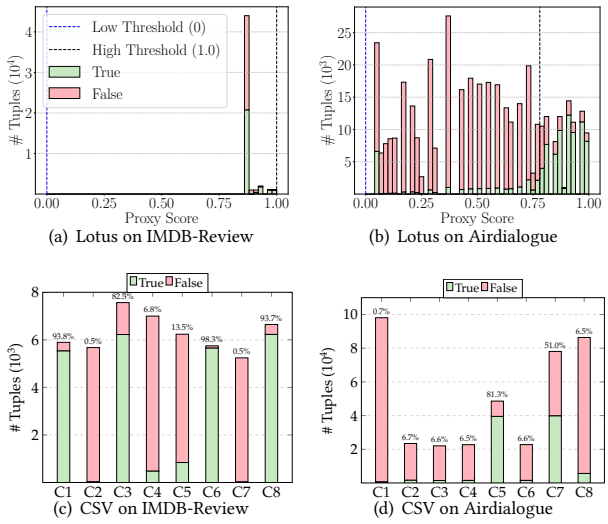


Figure 1: Data Label Distributions of Lotus and Ours (CSV)

To bridge the gap between semantic query processing and scalable data systems, many recent systems [4, 11, 25, 26, 34, 39–41, 44] have been proposed that enable users to specify semantic queries via DataFrame APIs or extended SQL with user-defined functions (UDFs). Despite differences in query expression and underlying system design, a coreset of semantic transformation operators can be 'distilled' from the queries they support. Analogous to the relational operators, Lotus [34] formulates a set of semantic operators, including *semantic filter, semantic map, semantic join*, etc., forming a declarative and composable query interface that enables operator-level algorithmic optimization. Among these operators, the semantic filter, serving as the counterpart of selection in relational algebra, is the cornerstone of queries in almost all systems. Given a table T with one or more textual attributes and a natural language (NL) predicate e, the semantic filter returns all tuples in T whose textual attributes satisfy the predicate e by LLM judgment. Users can use this operator to express a binary classification intrinsically. For example, in the IMDB-Review dataset [31], the

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

semantic filter with the NL predicate *'the review is positive'* can be used for sentiment analysis. Distinguished from relational operator evaluation, semantic operators combine the context of the NL predicate with the input tuple, together with the LLM's underlying knowledge learned from large corpora. Consequently, the outputs are inherently uncertain with respect to prompt structure, LLM backbone, and model configurations.

The conventional approach adopted by all existing systems for semantic filtering is to scan the table T linearly and issue an LLM invocation for each tuple. When the table is large, this approach incurs high query latency and substantial token consumption. To improve the query efficiency of the semantic filter, Lotus [34] introduces a two-stage model cascading algorithm, taking into account that LLM inference constitutes the bottleneck of query processing. The algorithm employs a lightweight proxy model, typically a small-sized LLM, to prefilter tuples and collect a fraction of tuples to learn thresholds for forwarding uncertain tuples to a more powerful LLM for final verification. However, in practice, we find that the cascading algorithm encounters dilemmas regarding both query accuracy and efficiency on real-world datasets. As illustrated in Fig.s 1(a) and 1(b), all tuples with proxy scores between the low and high thresholds must be forwarded to the powerful LLM. In Lotus, even in the best case, all data must be processed linearly by the proxy model. In the worst case, a poorly calibrated threshold causes nearly all data to be passed to the powerful LLM, negating the intended efficiency gains and exacerbating the overall inference cost. This issue is critical; however, optimization for the fundamental semantic filter operator has unfortunately been neglected.

To this end, we are motivated to reconsider optimizations for processing semantic filters at an algorithmic level, where a question arises: **can we reduce the complexity of LLM invocations while providing error guarantees for semantic filters?** For relational filtering, database systems construct B+-tree or Bitmap indices on the query attributes, replacing full table scans with local index lookups. For semantic filtering, although existing semantic analysis systems [28] allow users to build semantic indices using various pre-trained embedding models, it remains unclear whether the vectorized semantic indices can be leveraged for algorithm speedup and how to use them to reduce LLM invocations in semantic filtering. Furthermore, theoretical error analysis, which is highly intertwined with algorithm design, is more challenging but critical. Users typically treat LLMs as black boxes due to their sophisticated learning mechanisms, making query accuracy guarantees a necessary concern for system and algorithm designers. Despite the opacity of model belief from the deep learning perspective, a rigorous error guarantee from the filtering algorithm perspective helps interpret the query results and guide the configurations of the algorithm and system.

Our approach for optimizing semantic filtering is first inspired by recent progress in LLM inference for batch processing [29, 32, 36]. Given the high inference cost of LLMs, recent research has explored input caching strategies to accelerate online queries, where historical input queries together with their inference results are persisted in a cache. When a new query arrives, its semantically similar queries are retrieved from the cache, and their cached results are used directly as the proxy output. The intuition behind input caching for inference acceleration is that the more similar the input query, the more similar its output from LLMs. For a semantic filter query, the input of its LLM invocation is composed of three parts: a system prompt (i.e., a uniform instruction for semantic filtering), the predicate expression, and the value of a tuple, where the system prompt and predicate expression are identical for all the tuples for a given filter predicate. It is the tuple value that ultimately determines the results of LLM invocations, and the semantic similarity can be captured by clustering approaches.

In this paper, we propose a new paradigm dubbed *Clustering-Sampling-Voting (CSV)* for LLM-powered semantic filter processing, aiming to reduce the number of LLM invocations while preserving accuracy. The CSV framework is based on the observation that semantically similar inputs tend to elicit consistent outputs from LLMs. By exploiting this property, CSV amortizes inference costs across similar tuples while maintaining high labeling fidelity. The framework consists of three key phases: ① Clustering: All tuples in a relation are embedded using a pretrained encoder and grouped into semantically similar clusters using algorithms such as K-means. This step is performed offline and can be reused across queries. ② Sampling: Within each cluster, a small sample of tuples is drawn and evaluated with the LLM as representatives. ③ Voting: For the remaining tuples in each cluster, labels are inferred based on the LLM results from the sampled set, using either majority voting (Uniform Voting) or similarity-weighted voting (Similarity-based Voting). To ensure correctness, CSV provides theoretical guarantees that bound the discrepancy between the voting result and the expected LLM output. This is achieved by constraining the label frequency distribution within each cluster: if a cluster lacks semantic purity (i.e., labels are not well-separated), the framework recursively re-clusters the set of data into subsets until, in the worst case, it falls back to direct LLM evaluation for ambiguous subsets. Furthermore, our theoretical analysis explicitly bridges the expected error with the sample ratio, offering an intuitive way to balance theoretical assurance and LLM invocations. As illustrated in Fig.s 1(c) and 1(d), the label distributions across eight clusters in two datasets demonstrate that most clusters are dominated by a single label, validating the effectiveness of the clustering step. In rare cases, such as Cluster 7 in Fig. 1(d), where label purity is low, the framework triggers online re-clustering to preserve accuracy. This adaptive mechanism makes CSV both flexible and robust across diverse datasets and predicates.

The contributions of this paper are summarized as follows.

(1) **Algorithm Development.** We devise a new algorithm for LLM-powered semantic filtering that reduces the scale of LLM invocations to sublinear in the average case.
(2) **Theoretical Analysis.** We provide a detailed theoretical analysis of the proposed algorithm, which explicitly connects the error bound to the sample ratio.
(3) **Experimental Validation.** We conduct substantial experiments on multiple datasets and benchmarks to demonstrate the effectiveness and efficiency of our approaches. Our results show significant improvements in execution time, LLM call reduction, and token consumption compared to existing methods.

2 PRELIMINARIES & EXISTING APPROACHES

A semantic operator is a declarative transformation on relational data, which is parametrized by a natural language expression as a

predicate. In this paper, we consider the semantic filter powered by Large Language Models (LLMs).

2.1 Problem Statement

We use $T(A_1, \cdots, A_j)$, or T for short, to denote a table, where $A_1, \cdots, A_j$ are the attributes of T. $t \in T$ denotes a tuple in the table T and $t(A_j)$ denotes the value of the attribute A_j in the tuple t. $M : \mathcal{T} \mapsto \mathcal{T}$ is a pre-trained LLM that accepts input and generates output in textual space $\mathcal{T}$.

Semantic Filter. Given a natural language expression e, a semantic filter σ_M over one or multiple textual attributes of T by an LLM M, analogous to the relational selection operator, returns all the tuples in T that satisfy the semantic predicate defined by e. In the operator of Eq. (1), $M(t, e)$ denotes one LLM invocation, where the involved attributes in the tuple t, together with the expression e and an instruction, comprise the input of LLM, prompting the LLM to determine whether or not t can pass the semantic predicate e.

$$\sigma_{M(e)}(T) = \{t \in T \mid M(t, e) = \texttt{True}\}. \quad (1)$$

2.2 Existing Approaches and Issues

There are three existing approaches to semantic filtering in the algorithm aspect [26, 34, 50], with the exclusion of discussing other approaches due to their focus on system-level optimization over algorithmic enhancements [26, 39].

Reference Algorithm [26, 34, 50]: A straightforward algorithm to process a semantic filter is to invoke one LLM call $M(t, e)$ for each $t \in T$ sequentially or in parallel via table scan. The algorithm entails $O(|T|)$ LLM invocations in total, which is a huge cost.

Lotus [34]: To reduce the cost of per-tuple LLM evaluation, Lotus proposes a two-stage algorithm that uses a cheaper proxy LLM to avoid invoking an expensive oracle LLM on every tuple in T. For a semantic filter $\sigma_{M(e)}$, Lotus first samples a fixed number of tuples from T and learns two thresholds, τ_+ and τ_-, as decision boundaries of `True` and `False`, respectively. For each tuple $t \in T$, the proxy LLM is invoked to compute a proxy score, typically derived from the log-likelihood of LLM's output, which is available from open-source LLMs or certain APIs. If the proxy score is below τ_-, Lotus labels the result as `False`; if it exceeds τ_+, the result is marked as `True`; Tuples whose proxy scores fall in the ambiguous interval $[\tau_-, \tau_+]$ are forwarded to the more powerful oracle LLM for final verification.

Issues: Although Lotus provides an accuracy guarantee [19], its practical performance can degrade for several reasons. First, proxy cores are not always consistently well-aligned with the true labels. In favorable cases (e.g., Fig. 1(b) on Airdialogue), tuples spread across the score range and `True` tuples tend to receive higher scores than `False` tuples, making it easy to learn suitable thresholds that confidently label non-trivial fraction of tuples. However, in common cases (e.g., Fig. 1(a) on IMDB-Review), proxy scores concentrate in a narrow interval (i.e., $[0.8, 0.85]$), and `True` tuples do not reliably score higher than `False` tuples. This overlap makes threshold learning unstable from a small sample, and can yield degenerate thresholds, essentially reverting to Reference algorithm. Second, the sampling procedure used to fit τ_+ and τ_- can be mismatched to typical semantic filter workloads. The sampling strategy of Lotus is designed for scenarios requiring the extraction of low-selectivity tuples [22]. For semantic filters where labels are closer to balanced (as in Fig.s 1(a) and 1(b)), this strategy can produce a biased sample, which in turn drives the thresholds towards extreme values and reduces the proxy's ability to confidently rule out negatives. Third, the two-stage cascade algorithm does not break the bottleneck of linear LLM invocation and can even increase total cost. Lotus must invoke the proxy model for each tuple, which is still an LLM with considerable inference cost. When thresholds are poorly calibrated, a large fraction of tuples is forwarded to the oracle LLM, leading to an additional near-linear pass. In such cases, the two-stage cascade can exceed the cost of directly applying the Reference algorithm.

BARGAIN [50]: Similarly, BARGAIN begins by invoking a proxy LLM on each tuple $t \in T$ to obtain a proxy prediction and an associated proxy score. It then partitions tuples into score regions defined by pre-specified intervals (e.g., 20 score subranges). BARGAIN starts sampling from the region with the highest proxy scores and progressively expands to lower-score regions. Within each region, it samples tuples and invokes the oracle LLM to test whether the tuples in that region can meet a user-specified quality target. The region-wise procedure terminates once BARGAIN identifies a minimum proxy-score threshold such that including any lower-score region would violate the target quality. Finally, tuples whose proxy scores fall below this threshold are forwarded to the oracle LLM for final verification.

Issues: Since BARGAIN requires scanning the dataset with a substantial proxy LLM, akin to Lotus, it needs a considerable overhead on this linear processing. Importantly, recent studies [15, 16] indicate that neural networks frequently yield poorly calibrated confidence scores, inaccurately representing true correctness probabilities due to their inherent overconfidence of neural networks. This issue could potentially impact BARGAIN's performance, as the proxy score forms the foundation of its approach.

3 OUR APPROACH FOR SEMANTIC FILTER

In this section, we present a new semantic filter algorithm based on a clustering-sampling-voting (CSV) paradigm, designed to address the issues discussed in §2. At a high-level, our approach replaces per-tuple proxy LLM calling with embedding-based grouping. We compute an embedding for each tuple using an embedding model; these embeddings can be generated offline (e.g., at table creation time or upon tuple insertion) and are substantially cheaper than invoking an LLM to obtain proxy scores. We cluster tuple embeddings and infer a label for each cluster based on whether `True` or `False` is dominated within the cluster. As illustrated in Fig.s 1(c) and (d), this enables labeling a large number of tuples without an LLM invocation.

Our CSV paradigm builds on the intuition that embedding-similar tuples tend to elicit similar responses from LLMs; thereby, embedding-based grouping can serve as an initial proxy. This intuition is commonly exploited in inference caching: when two prompts are close in semantic space, their outputs are likely to be similar in meaning, enabling the reuse of prior inference results [29, 32, 36]. In semantic filtering, the system prompt and predicate expression are fixed, and only the tuple content varies across invocations. Consequently, variability in the LLM's output is fully driven by the tuple, and

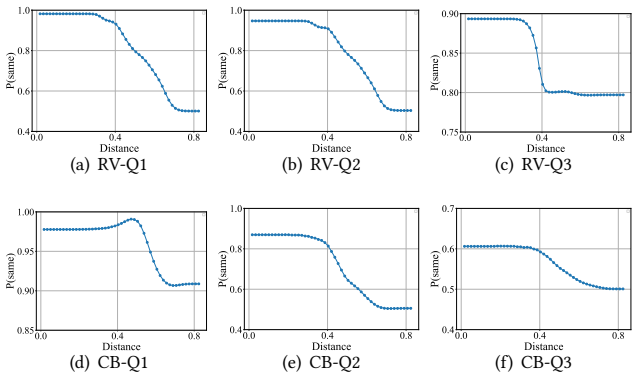


Figure 2: Probability Distribution Across Distance

semantic proximity between tuples can be captured by distances in an embedding space.

We empirically validate this assumption by quantifying the relationship between embedding distance and label agreement across multiple predicates. For each tuple, we compute its embedding using E5-Large [45] and obtain from the LLM the probability of the `True` label. For each tuple pair (t_i, t_j), we compute the Euclidean distance between their embeddings and the probability that the two tuples share the same label of `True` or `False`. By splitting the range of all-pair distances into 50 bins, Fig. 2 illustrates the trends of the mean probability of label agreement as the distances increase across 6 queries on two datasets. Across predicates, the results show a consistent pattern: as embedding distance increases, the probability of label agreement decreases. The magnitude of the effect varies by predicate—e.g., RV-Q1, RV-Q2, and CB-Q2 exhibit a sharper decline, whereas RV-Q3 and CB-Q1 decrease more gradually—but the overall trend remains stable. These findings provide quantitative support for using embedding-based clustering as a mechanism to amortize LLM evaluations in semantic filtering. In the following, we present the details of CSV in §3.1 and provide its theoretical analysis in §3.2.

3.1 A Clustering-Sampling-Voting Paradigm

As this name indicates, the Clustering-Sampling-Voting (CSV) paradigm processes semantic filter queries in two phases: an offline clustering phase and an online sampling with voting phase. In the offline phase, we employ a clustering algorithm such as K-means to partition the tuples in the table T into clusters, which is query-agnostic. In the online phase, we sample a subset of tuples from each cluster, invoke LLM to evaluate these sampled tuples regarding the semantic predicates, and ensemble their results via voting to determine the remaining tuples in the cluster. As the voting mechanism will directly bypass LLM invocations for unsampled tuples, to preserve high accuracy, we use two thresholds ub and lb to limit this direct filtering for tuples with low confidence. Also, in the following theoretical analysis part, we prove that the error is guaranteed by thresholds ub and lb. The algorithm initiates a fallback mechanism for these low-confident tuples, resorting to online fine-grained re-clustering, re-sampling, and re-voting to refine decisions. If uncertainty persists and the recursive depth reaches its maximum threshold, the framework ultimately falls back to direct LLM invocation for the remaining ambiguous tuples.

Algorithm 1: SemanticFilter(T, e, M, k, ξ)

Input : input table T, expression e, LLM M, #clusters k, and sample ratio ξ
Output : output table R

1 Initialize $R \leftarrow \emptyset$;
2 Generate the embedding $\mathbf{e}_i$ for each tuple $t_i \in T$;
3 Partition $\{(\mathbf{e}_1, t_1), \cdots, (\mathbf{e}_n, t_n)\}$ into k clusters $C = \{C_1, \cdots, C_k\}$ by k-means over $\{\mathbf{e}_1, \cdots, \mathbf{e}_n\}$;
4 Initialize $Q \leftarrow C$;
5 **while** Q *is not empty* **do**
6 Initialize $\mathcal{U} \leftarrow \emptyset$;
7 **for** $C_j \in Q$ **do**
8 Q.remove(C_j) ;
9 Randomly sample subset $C'_j \subset C_j$ with ratio ξ ;
10 Allocate $O \leftarrow \emptyset$;
11 **for** $(\mathbf{e}_i, t_i) \in C'_j$ **do**
12 $o_i \leftarrow M(t_i, e)$;
13 $O \leftarrow O \cup \{o_i, t_i, \mathbf{e}_i)\}$;
14 **if** o_i *is* `True` **then**
15 $R \leftarrow R \cup \{t_i\}$;
16 $C^*, R^+ \leftarrow$ Vote(O, C_j, C'_j) ;
17 $R \leftarrow R \cup R^+$;
18 **if** C^* *is not empty* **then**
19 $\mathcal{U} \leftarrow \mathcal{U} \cup C^*$;
20 **if** C^* *is not empty* **then**
21 Partition $\mathcal{U}$ into new clusters $\{C_{j_1}, \cdots, C_{j_m}\}$ by k-means ;
22 $Q \leftarrow \{C_{j_1}, \cdots, C_{j_m}\}$;
23 Return R ;

Algorithm 1 presents the procedure of CSV for the semantic filter, given a table T, a semantic expression e, an LLM M, and a parameter for the number of clusters k. In the offline phase, the algorithm adopts an embedding model to encode each tuple t_i into a vector representation $\mathbf{e}_i$ (line 2), and then partitions the tuples into k clusters $\{C_1, \cdots, C_k\}$ by K-means regarding the semantic similarity of the embeddings (line 3). In the online sampling and voting phase, we process each cluster individually by voting strategy. Here, we use a set Q to maintain all the clusters to be processed. For each cluster layer, we initialize a temporary set $\mathcal{U}$ (line 8) to collect all low-confidence tuples that require further refinement. For a cluster C_i, we first draw a collection of tuples, C'_i, uniformly with a sampling rate ξ (line 9). Afterwards, we invoke the LLM M to infer the result for each sampled tuple, respectively, regarding the semantic predicate e (lines 10-15), where the results are persisted in a temporary set O. Sampled tuples that pass the LLM-based semantic filter are directly inserted into the result table R (lines 14-15). For the remaining tuples in $C_j \setminus C'_j$, the algorithm introduces the voting mechanism, i.e., the function Vote in line 16, which takes the results of the sampled tuples, all tuples in the current cluster, and the sampled tuples as inputs. The function returns two

Algorithm 2: UniVote(O, C, C')

Input : a set of sampled tuples and LLM invocation results O, a cluster of tuples with their embedding C, the sample subset of tuples with their embeddings C'

Output : undetermined tuples with their embedding C^* and positive tuples R^+

1 $O^+ \leftarrow \{o_i = \texttt{True} \mid (o_i, t_i, \mathbf{e}_i) \in O\}$;
2 score $\leftarrow \frac{|O^+|}{|O|}$;
3 **if** *score* $\geq$ *upperbound* **then**
4 $\quad C^* \leftarrow \emptyset, R^+ \leftarrow \{t_i \mid (t_i, \mathbf{e}_i) \in C \setminus C'\}$;
5 **else if** *score* $\leq$ *lowerbound* **then**
6 $\quad C^* \leftarrow \emptyset, R^+ \leftarrow \emptyset$;
7 **else**
8 $\quad C^* \leftarrow C \setminus C', R^+ \leftarrow \emptyset$;
9 return C^*, R^+ ;

sets of tuples, C^* and R^+, which denote the low-confident tuple set that needs fallback verification and the high-confident tuple set that is determined to pass the filter by voting, respectively. We append R^+ to the result table line 17. For the low-confidence tuples, we collect all such sets into $\mathcal{U}$ (line 18). After finishing the current layer, if U is nonempty, we repartition $\mathcal{U}$ as a whole by K-means (line 19) and push the resulting subclusters back into the set Q for the next iteration (line 20). Each new sub-cluster is then processed following the same procedure (lines 10-19), i.e., uniform sampling and voting, until a maximum split limit is reached. We equip two voting strategies for the clustering & voting algorithms, i.e., uniform voting (Algorithm 2) and similarity-based voting (Algorithm 3). Both of these strategies use two thresholds $ub, lb \in (0, 1)$ to evaluate the voting belief. We elaborate on these two algorithms in the following.

Uniform Voting. Given the results of the sampled tuples O, Uniform Voting computes the ratio of tuples that pass the filter (line 1 in Algorithm 2). Comparing it with the two thresholds ub and lb results in three cases as shown in Eq. (3).

$$\text{score}(t_i) = \frac{|O^+|}{|O|}, \forall t_i \in C \setminus C', \tag{2}$$

$$\hat{M}(t_i; e) = \begin{cases} \text{True} & \text{if score}(t_i) \geq ub \\ \text{False} & \text{if score}(t_i) \leq lb \\ \text{Undetermined} & \text{if score}(t_i) \in (lb, ub) \end{cases}. \tag{3}$$

Here, we use $\hat{M}(t, e)$ to denote the vote result on a tuple t regarding e. Under each case, Uniform Voting will return corresponding tuples which are regarded as passing the filter, R^+, and the undetermined tuples C^* (lines 3-8). In Uniform Voting, the result of sampled tuples in C' contributes equally, and the voting results $\hat{M}(t; e)$ are equal for all the remaining tuples in the cluster. To achieve fine-grained decision making, we further propose a similarity-based voting mechanism that takes the embedding similarity between sampled tuples and remaining tuples into account.

Similarity-based Voting. Algorithm 3 presents the process of Similarity-based Voting, which also returns the two sets R^+ and C^*, as additional results and undetermined tuples. In Algorithm 3, for

Algorithm 3: SimVote(O, C, C')

Input : a set of sampled tuple and LLM invocation results O, a cluster of tuples with their embedding C, the sample subset of tuples with their embeddings C'

Output : undetermined tuples with their embedding C^* and positive tuples R^+

1 $C^* \leftarrow \emptyset, R^+ \leftarrow \emptyset$;
2 **for** $(t_j, \mathbf{e}_j) \in C \setminus C'$ **do**
3 $\quad$ score $\leftarrow 0$;
4 $\quad$ norm $\leftarrow \sum_{t_k \in C'} \text{sim}(e_j, e_k)$;
5 $\quad$ **for** $(o_i, t_i, \mathbf{e}_i) \in O$ **do**
6 $\quad\quad$ flag $\leftarrow 1$ if o_i is True else 0 ;
7 $\quad\quad$ score $\leftarrow$ score $+ \frac{\text{sim}(\mathbf{e}_i, \mathbf{e}_j)}{\text{norm}} \cdot$ flag ;
8 $\quad$ **if** *score* $>$ *upperbound* **then**
9 $\quad\quad R^+ \leftarrow R^+ \cup \{t_j\}$;
10 $\quad$ **else if** *score* $<$ *upperbound* $\wedge$ *score* $>$ *lowerbound* **then**
11 $\quad\quad C^* \leftarrow C^* \cup \{(t_j, \mathbf{e}_j)\}$;
12 return C^*, R^+ ;

each remaining tuple t_i, we aggregate the voting results of each sampled tuple t_j weighted by its normalized similarity between t_i, and the score function is reformulated as Eq. (4)

$$\text{score}(t_i) = \sum_{t_j \in C'} \frac{\text{sim}(e_i, e_j)}{\sum_{t_k \in C'} \text{sim}(e_i, e_k)} \mathbb{I}(M(t_j, e)), \forall t_i \in C \setminus C', \tag{4}$$

where $\mathbb{I}(M(t_j, e))$ is the identity function of LLM invocations on the sampled tuples. Similarly, the score function is compared with lb and ub following Eq. (3).

Compared to Uniform Voting, Similarity-based Voting can provide robustness when initial clustering does not perform well. Specifically, when the distribution of positive and negative labels within a cluster does not satisfy our criteria, i.e., lb and ub, Uniform Voting will re-cluster the entire cluster, whereas Similarity-based Voting may vote successfully on certain items. This reduces the amount of unprocessed data in subsequent steps. For example, the results of AD-Q1 and AD-Q4 shown in § 4.2, Similarity-based Voting outperforms Uniform Voting, in reducing LLM calls.

Example 3.1. (Clustering-Sampling-Voting). Fig. 3 illustrates a running example of Clustering & Uniform Voting on IMDB-Review dataset. The dataset consists of a table of tuples, each representing a movie review. The semantic filter query applied here is "The review is positive." We first partition the table into 8 clusters using K-means clustering based on the tuple embeddings. The label distributions of each cluster are depicted in Fig. 1(c). Next, voting is conducted independently for each cluster. We use clusters C3 (highlighted in green) and C8 (highlighted in blue) as examples. In this running example, we set $lb = 0.15$ and $ub = 0.85$. In cluster C8, 101 tuples are uniformly sampled, and their semantic filter results are obtained from individual LLM invocations, e.g., the semantic filter yields True for the tuple "this file is outstanding...". Among the sampled tuples, 99.01% are labeled as True, surpassing the upper bound ($0.9901 > ub$), leading us to categorize all tuples in this cluster as True. For cluster C3, where 101 tuples are uniformly sampled,

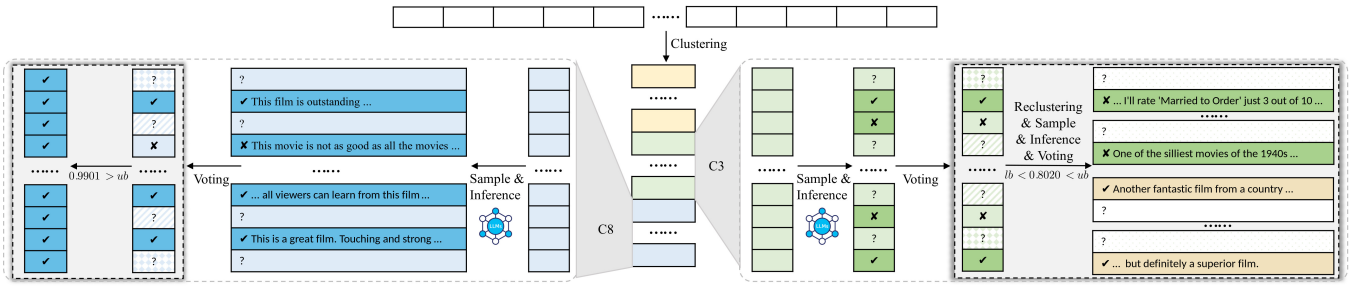


Figure 3: An Running Example of Clustering-Sampling-Voting (CSV) with UniVote on IMDB-Review Dataset

80.20% are classified as `True`. Since $lb < 0.8020 < ub$, we apply a re-cluster strategy. This strategy results in two clusters, distinguished by green and yellow colors in the far right of Fig. 3. Similarly, we sample tuples from each of these two clusters, utilize LLM for individual tuple analysis, and subsequently conduct voting.

Complexity Analysis. Given the table T and a sampling rate of ξ, the CSV paradigm issues $O(\xi|T|)$ LLM invocations in the best case and issues $O(|T|)$ LLM invocations in the worst case.

The CSV framework can support filters on multiple columns by generating a fused semantics embedding for multiple columns. For example, concatenating the text of multiple columns together with the column names and feeding the long context into a pre-trained language model to produce the row-wise embedding, or indexing the embeddings for all columns by hierarchical clustering. In addition, our approach can be extended to support tuple insert/delete by incorporating incremental clustering algorithms to maintain the clusters. Our update handling distinguishes between small and large batches. (1) Small, occasional updates. When the update batch is small, the cluster centroids are assumed to be unchanged. New data are inserted into the clusters with the nearest centroid, and we reuse the existing cluster-level voting outcomes. If a cluster previously had no consensus, we apply linear LLM invocations to incorporate the minor updates while keeping the overhead low. (2) Larger, periodic updates. For larger batches of updates, we use mini-batch K-means to update the clustering. Subsequent sampling, voting, and re-clustering are fully compatible with the underlying data. We cache the LLM outcomes for sampled tuples from the original datasets for potential reuse. For samples in the updated datasets, LLM is invoked to process tuples missed in the cache. For deletion, deleted tuples will be marked, and clusters will be merged when the number of them reaches a specific number.

3.2 Analysis on ξ Setting of Voting Strategies

§3.1 introduces two voting strategies, UniVote and SimVote. Both strategies are governed by a key parameter, the sample ratio ξ, which directly impacts both the time complexity (requiring $O(\xi|T|)$ LLM invocations) and the theoretical error (as elaborated in the subsequent discussion). Concretely, from a given cluster, the voting strategies sample a subset of tuples, constituting a proportion ξ of the entire tuples in the cluster, invoke the LLM on the sampled tuples, and use the sampled outcomes to infer the cluster-level label. The choice of ξ induces a natural trade-off. A larger ξ increases the number of LLM calls but reduces estimation error; conversely, a smaller ξ reduces LLM invocations with the compromise of a higher error. In practice, however, users typically lack the prior knowledge needed to choose ξ to balance cost and quality.

To make principled parameter tuning, we introduce a user-friendly parameter, ϵ, representing a user-specified error tolerance in our theoretical analysis. In the following, we derive a theoretical relationship between ξ and ϵ, enabling us to select an appropriate sampling ratio ξ for a given ϵ, so that our CSV framework achieves the desired accuracy with high probability.

First, we introduce the cornerstone of our theoretical analysis, Bernstein Inequality [6] in Lemma 3.2, which will be frequently used subsequently.

Lemma 3.2. *(Bernstein Inequality [6]) Given $X = \{x_1, \cdots, x_n\}$ is the finite population of size n, where $x_1, \cdots, x_n$ are binary variables with a mean μ. Let $\hat{\mu}$ be the mean of k variables $c_1, \cdots, c_k$ obtained by simple random sampling without replacement from X. For any $\epsilon > 0$, we have*

$$Pr\left[|\hat{\mu} - \mu| \geq \epsilon\right] \leq 2exp\left(-\frac{k\epsilon^2}{2\hat{\sigma}^2 + 2R\epsilon/3} \cdot \frac{n-k}{n-1}\right),$$

where $\hat{\sigma}$ is the sample standard deviation of k sampled variables $c_1, \cdots, c_k$, i.e., $\hat{\sigma}^2 = \frac{1}{k-1}\sum_{i=1}^{k}(c_i - \hat{\mu})^2$, R is the almost sure bound for x_i, which means $|x_i| \leq R$ almost surely.

Analysis of UniVote. First, we analyze the SemanticFilter with UniVote. Here, given $X = \{x_1, \cdots, x_n\}$ of C_j, where $x_1, \cdots, x_n$ are binary, and $x_i = \mathbb{I}(M(t_i, e))$ for each $(\mathbf{e}_i, t_i) \in C_j$. Let $\mu(X)$ be the mean of variables in X, and $f(t_i) \in \{0, 1\}$ indicate whether $(\mathbf{e}_i, t_i)$ is included in R^+ by UniVote. Specifically, $f(t_i) = 1$ if $|O^+|/|O| \geq ub$ and $f(t_i) = 0$ if $|O^+|/|O| \leq lb$, where $|O| = k$ and $\xi = |O|/|C_j|$. By the Bernstein bound, we confirm that $f(t_i)$ is an accurate estimator of x_i for each $(\mathbf{e}_i, t_i)$ when the sampling rate ξ is sufficiently large.

Theorem 3.3. *Given a cluster $C = \{(\mathbf{e}_1, t_1), \cdots, (\mathbf{e}_n, t_n)\}$ and parameters lb, ub, l and ϵ. Let $X = \{x_1, \cdots, x_n\}$ of C, where $x_i = \mathbb{I}(M(t_i, e))$ is binary for each $(\mathbf{e}_i, t_i) \in C$. Assume that ξ satisfies*

$$\xi \geq \frac{1}{2} - \sqrt{\frac{1}{4} + \ln l\left(\frac{2\hat{\sigma}^2}{\epsilon^2} + \frac{2}{3\epsilon}\right)}.$$

Then, the following inequality holds with at least $1 - 2l^n$ probability when $\mu(O) \leq lb$ or $\mu(O) \geq ub$:

$$Pr[f(t_i) \neq x_i] \leq \max(lb + \epsilon, 1 - (ub - \epsilon)).$$

Theorem 3.3 bounds the probability that the voting decision $f(t_i)$ disagrees with the true LLM output x_i. The key insight is that the mean of samples $\mu(O)$ concentrates around the mean of the true population $\mu(X)$ with high probability. The condition on ξ ensures that we sample enough tuples for the Bernstein inequality to guarantee this concentration within tolerance ϵ. Consequently, if voting assigns a label, the error is bounded by $\max(lb+\epsilon, 1-(ub-\epsilon))$, and this guarantee holds with probability at least $1-2l^n$.

Proof Sketch: Let $\mu(O)$ be the mean of k sampled elements from X, i.e., $k=\xi n \geq \frac{1}{2}n - n\cdot\sqrt{\frac{1}{4}+\ln l\left(\frac{2\hat{\sigma}^2}{\epsilon^2}+\frac{2}{3\epsilon}\right)}$ and $\mu(O)=|O^+|/|O|$.

For the case that $\mu(O)\leq lb$, we have $f(t_i)=0$. Hence, $Pr[f(t_i)\neq x_i]=Pr[x_i\neq 0]=Pr[x_i=1]=\mu(X)$. Then we have $Pr[Pr[f(t_i)\neq x_i]\geq lb+\epsilon]=Pr[\mu(X)\geq lb+\epsilon]\leq Pr[\mu(X)\geq \mu(O)+\epsilon]\leq Pr[|\mu(X)-\mu(O)|\geq\epsilon]$. According to Lemma 3.2, we have $Pr[|\mu(X)-\mu(O)|\geq\epsilon]\leq 2\exp\left(-\frac{k\epsilon^2}{2\hat{\sigma}^2+2\epsilon/3}\cdot\frac{n-k}{n-1}\right)<2\exp(n\ln l)=2l^n$.

For another case that $\mu(O)\geq ub$, we have $f(t_i)=1$. Hence, $Pr[f(t_i)\neq x_i]=Pr[x_i\neq 1]=Pr[x_i=0]=1-\mu(X)$. Then we have $Pr[Pr[f(t_i)\neq x_i]\geq 1-(ub-\epsilon)]=Pr[1-\mu(X)\geq 1-(ub-\epsilon)]=Pr[\mu(X)\leq ub-\epsilon]\leq Pr[\mu(X)\leq\mu(O)-\epsilon]\leq Pr[|\mu(X)-\mu(O)|\geq\epsilon]$. Based on the preceding analysis, we have already established that $Pr[|\mu(X)-\mu(O)|\geq\epsilon]\leq 2l^n$. □

Note that, in the scenario where $lb<|O^+|/|O|<ub$, the SemanticFilter continues to repartition the cluster while maintaining the values of lb and ub. Therefore, the analysis remains consistent. Therefore, here, we only focus on cases where $\mu(O)=|O^+|/|O|$ satisfies either $\mu(O)\leq lb$ or $\mu(O)\geq ub$.

Analysis of SimVote. Next, we analyze the SemanticFilter with SimVote. We consider a weighted version of Lemma 3.2.

Corollary 3.4. *Let $X=\{x_1,...,x_n\}$ be a finite population of size n, where $x_1,\cdots,x_n$ are binary variables with a mean μ. Let $c_1,\cdots,c_k$ be sampled without replacement from X, and each c_i is associated with a weight w_i, where $w_i\geq 0$ and $\sum_{i=1}^k w_i=1$. Let $\hat{\mu}$ be the mean of $c_1,\cdots,c_k$, i.e., $\hat{\mu}=\frac{1}{k}\sum_{i=1}^k c_i$; and $\hat{\mu}_w$ be the weighted mean of c_i with weight w_i, i.e., $\hat{\mu}_w=\frac{1}{k}\sum_{i=1}^k w_i c_i$. For any $\epsilon>0$, we have*

$$Pr[|\hat{\mu}_w-\mu|\geq\epsilon]\leq 2\exp\left(\frac{-3k\epsilon^2}{(6\hat{\sigma}^2+2\epsilon)v}\cdot\frac{n-k}{n-1}\right),$$

where $\hat{\sigma}^2=\frac{1}{k-1}\sum_{i=1}^k(c_i-\hat{\mu})^2$ is the sample standard deviation, and v is a constant such that $\max_i w_i\leq\frac{v}{k}$.

The derivation can be written as $\hat{\mu}_w-\mu=\sum_{i=1}^k y_i$ with $y_i=w_i(c_i-\mu)$, whose expectation is 0. Applying Lemma 3.2 to y_i and bounding the variance, Corollary 3.4 can be proved. Full derivations and detailed proof steps are provided in our technical report [1].

Here, given $X=\{x_1,\cdots,x_n\}$ of a cluster C_j, where $x_1,\cdots,x_n$ are binary, and $x_i=\mathbb{I}(M(t_i,e))$ for each $(\mathbf{e}_i,t_i)\in C_j$. Let $g(t_i)\in\{0,1\}$ indicate whether $(\mathbf{e}_i,t_i)$ is included in R^+ by SimVote, i.e., $g(t_i)=1$ if $\text{score}(t_i)\geq ub$ and $g(t_i)=0$ if $\text{score}(t_i)\leq lb$, where $\text{score}(t_i)$ is defined in Eq. (4).

Lemma 3.5. *Given a cluster $C=\{(\mathbf{e}_1,t_1),\cdots,(\mathbf{e}_n,t_n)\}$ and its subset $O\subseteq C$, which is sampled uniformly from C. Let $X(C)=\{x_1,\cdots,x_n\}$ of C, where $x_i=\mathbb{I}(M(t_i,e))$ is binary for each $(\mathbf{e}_i,t_i)\in C$, and $\mu(score(C\setminus O))$ be the mean value of $score(t_i)$ (defined in Eq. 4) for $(\mathbf{e}_i,t_i)\in C\setminus O$. We have $\mu(score(C\setminus O))=\mu(X(O))$.*

Proof Sketch: Since $\mathbb{I}(M(t_j,e))=x_j$, we have

$$|C\setminus O|\cdot\mu(\text{score}(C\setminus O))=\sum_{t_i\in C\setminus O}\sum_{t_j\in O}\frac{\text{sim}(t_i,t_j)}{\sum_{t_k\in O}\text{sim}(t_i,t_k)}x_j$$
$$=\sum_{t_j\in O}x_j\cdot\left(\sum_{t_i\in C\setminus O}\frac{\text{sim}(t_i,t_j)}{\sum_{t_k\in O}\text{sim}(t_i,t_k)}\right). \quad (5)$$

Since O is sampled uniformly from C, we have r.h.s. of Eq. (5) $=\sum_{t_j\in O}x_j|C\setminus O|$, hence, $\mu(\text{score}(C\setminus O))=\sum_{t_j\in O}x_j=\mu(X(O))$. □

Based on Corollary 3.4 and Lemma 3.5, we confirm that $g(t_i)$ is an accurate estimator of x_i for each $(\mathbf{e}_i,t_i)$ when the sampling rate ξ is sufficiently large.

Theorem 3.6. *Given a cluster $C=\{(\mathbf{e}_1,t_1),\cdots,(\mathbf{e}_n,t_n)\}$ and parameters lb, ub, l, v and ϵ. Let $X=\{x_1,\cdots,x_n\}$ of C, where $x_i=M(t_i,e)$ is binary for each $(\mathbf{e}_i,t_i)\in C$. Assume that ξ satisfies*

$$\xi\geq\frac{1}{2}-\sqrt{\frac{1}{4}+\frac{vlnl(6\hat{\sigma}^2+2\epsilon)}{3\epsilon^2}}$$

Then, the following inequality holds with at least $1-2l^n$ probability when $score(t_i)\leq lb$ or $score(t_i)\geq ub$:

$$Pr[g(t_i)\neq x_i]\leq\max(lb+\epsilon,1-(ub-\epsilon)).$$

Theorem 3.6 extends a theoretical guarantee to SimVote. Although each tuple receives a personalized score, Lemma 3.5 ensures that these scores concentrate around the same mean as uniform sampling. The condition on ξ in Theorem 3.6 is slightly more complex than in Theorem 3.3 due to the variance derived from the weighting mechanism. Despite this, the final guarantee takes the same form.

Proof Sketch: We have $k=\xi n\geq n/2-\sqrt{n^2/4+n^2vlnl(6\hat{\sigma}^2+2\epsilon)/3\epsilon^2}$. For the case $score(t_i)\leq lb$, we have $g(t_i)=0$. Hence, $Pr[g(t_i)\neq x_i]=Pr[x_i\neq 0]=Pr[x_i=1]=\mu(X)$. Then $Pr[Pr[g(t_i)\neq x_i]\geq lb+\epsilon]=Pr[\mu(X)\geq lb+\epsilon]\leq Pr[\mu(X)\geq score(t_i)+\epsilon]\leq Pr[|\mu(X)-score(t_i)|\geq\epsilon]$. According to Lemma 3.5, we have $\mu(score(t_i))=\mu(X)$ and based on Corollary 3.4, we have $Pr[|\mu(X)-score(t_i)|\geq\epsilon]\leq 2exp(\frac{-3k\epsilon^2}{(6\hat{\sigma}^2+2\epsilon)v}\cdot\frac{n-k}{n-1})<2\exp(n\ln l)=2l^n$. For another case that $score(t_i)\geq ub$, we have $g(t_i)=1$. Hence, $Pr[g(t_i)\neq x_i]=Pr[x_i\neq 1]=Pr[x_i=0]=1-\mu(X)$. Then we have $Pr[Pr[g(t_i)\neq x_i]\geq 1-(ub-\epsilon)]=Pr[1-\mu(X)\geq 1-(ub-\epsilon)]=Pr[\mu(X)\leq ub-\epsilon]\leq Pr[\mu(X)\leq score(t_i)-\epsilon]\leq Pr[|\mu(X)-score(t_i)|\geq\epsilon]$. Based on the preceding analysis, we have already established that $Pr[|\mu(X)-score(t_i)|\geq\epsilon]\leq 2l^n$.

4 EXPERIMENTAL STUDY

In this section, we present the experimental setup in §4.1 and evaluate our approaches comprehensively in the following facets: ❶ Compare the baselines on the semantic filter and present the results of overall effectiveness and efficiency in §4.2. ❷ Analyze the effects of hyper-parameters on CSV in §4.3. ❸ Investigate the efficacy of re-clustering in CSV in §4.4. ❹ Explore the disparity between practical and theoretical results in §4.5. ❺ Evaluate the generalizability of CSV on diverse synthetic queries in §4.6. ❻ Study the impacts of employing different embedding models and LLMs in our technical report [1].

4.1 Experimental Setup

Datasets. Following [7, 28, 34, 50] we evaluate five datasets for semantic filter. ① **IMDB-Review:** The IMDB-Review [31] dataset is a binary sentiment classification benchmark derived from the movie review domain. It comprises 50,000 labeled reviews, which are full-length user-written reviews from Internet Movie Database (IMDB), each expressing either a positive or negative opinion about a film. The dataset is evenly split between the two sentiment classes. ② **Codebase:** The Codebase dataset [24] consists of a structured table named user with 9,378 rows, which includes a column titled AboutMe containing long-form self-introduction texts. ③ **Airdialogue:** The Airdialogue dataset [46] is a classification task derived from the airline ticketing domain. It comprises 402,035 unique dialogs, each annotated with one of five possible categories: book (51.40%), cancel (1.46%), no_flight (23.08%), no_reservation (23.89%), and change (0.17%). ④ **TC:** The Twitter Hate Speech and Offensive Language (TC) dataset [12] is a classification benchmark derived from the social media domain. It comprises 24,783 labeled tweets, each annotated into one of three categories: hatespeech, offensivelanguage, or neither. The dataset was collected from Twitter using hate speech-related keywords and subsequently annotated via crowdsourcing. ⑤ **Fever:** The Fact Extraction and Verification (Fever) dataset [42] is a large-scale benchmark for claim verification in the fact-checking domain, constructed by altering sentences extracted from Wikipedia. In this work, we use a cleaned version from which incomplete entries have been removed to ensure consistency and reliability.

Queries. Table 1 summarizes the 14 test queries used in our experiments. For RV-Q2/Q3 and CB-Q1/Q2/Q3, ground-truth labels are not publicly available; we therefore use GPT-4o as a proxy to obtain these labels via direct LLM calls. Among the 12 queries, Fever is a multi-column semantic filter that references two columns, the remaining queries are single-column predicates. The task on Airdialogue is a multi-class classification problem that determines the category of a dialog. We decompose the task into four binary semantic filter queries, AD-Q1/Q2/Q3/Q4, where each query is a predicate for one of the four classes: book/cancel/no_flight/no_reservation. In addition to predicates shown in Table 1, we prepend a following instruction to the prompt for all Airdialogue queries: 'The following is a dialog between an airline ticketing agent and a customer. The outcome of the dialog will be one of the following 5 categories. [book]: the agent has booked a flight for the customer (not including the flight change). [cancel]: the agent canceled the existing valid reservation for the customer. [change]: the agent changed the existing flight reservation of the customer and successfully found a new one. [no_reservation]: the customer wants to change or cancel the flight, but there is no valid reservation under this customer. [no_flight]: the customer aims to book a flight from departure to destination but finds no flights between departure and destination.'

Algorithms. We compare our approaches with the following existing methods. ① **Reference**: Following the definition in Eq. (1), we sequentially invoke the LLM for each tuple and retrieve the results to establish a baseline for the semantic filter. ② **Lotus**: Lotus introduces state-of-the-art approaches for the semantic filter [34]. We adopt the approach as a baseline for the respective tasks. ③ **BARGAIN**: BARGAIN [50] adopts a similar model cascade framework as Lotus while providing strong theoretical guarantees on Precision, Recall, and Accuracy. We configure BARGAIN with an accuracy target of 0.85 and a tolerance of 0.05. For both Lotus and BARGAIN we use the default hyper-parameters provided in their respective source code repositories. Both methods use LlaMA-3.2-3B for initial filtering and LlaMA-3.1-8B as the larger model for secondary processing.

For our approaches, ⑤ **UniCSV** and ⑥ **SimCSV** are two algorithms for the semantic filter (Algorithm 1), instantiated with UniVote (Algorithm 2) and SimVote (Algorithm 3), respectively. By default, we employ LLaMA-3.1-8B [43] as the backbone LLM and E5-Large [45] as the embedding model. When input text exceeds the context length supported by the embedding model, it is segmented into smaller chunks, each embedded independently. The final embedding is computed as the mean of all segment embeddings to ensure a consistent representation across variable-length inputs. For clustering, we employ the K-means algorithm from [35] and set the default number of clusters to 4. To prevent excessive re-clustering during the filtering process, the maximum number of re-cluster iterations is capped at 3. Once a voting failure occurs in the remaining clusters, we resort to sequential LLM invocation to ensure bounded errors. To better accommodate the diversity of predicates, we further introduce BM25 score [17] that captures lexical similarity, as a distance metric for clustering, used jointly with the embedding Euclidean distance. A hyper-parameter λ is introduced to balance the relative contributions between distances, i.e., λ for L2 distance and $(1-\lambda)$ for BM25. We set $\lambda = 0.4$ for CB-Q1 and TC, and $\lambda = 1.0$ for the remaining queries. Other hyper-parameters are as follows, $lb = 0.15$, $\xi = 0.005$, min_sample = 101.

Evaluation Metrics. We evaluate different methods by Accuracy, Recall, F1 score; and efficiency by execution time, # LLM calls, and token consumption. This multifaceted evaluation enables a balanced analysis of both predictive effectiveness and computational efficiency. For efficiency, execution time measures the end-to-end wall-clock time required to complete the semantic operator. # LLM call refers to the total number of invocations of the LLMs. For consistency, we count calls based on the number of prompts issued, irrespective of whether multiple tuples are batched within a single prompt. Token consumption captures the total number of tokens processed, including both input (prompt) and output tokens.

Implementation Details The experiments are conducted locally on a single 80GB NVIDIA A100 GPU, with vLLM [23] serving as the inference engine. For all approaches, batch prompt processing is handled using the *batch_completion* method from LiteLLM [2], with a temperature of 0.7 and a maximum output token limit of 32.

4.2 Overall Evaluation of Semantic Filter

In this section, we compare our two approaches, UniCSV and SimCSV, against the baselines on both efficiency and effectiveness.

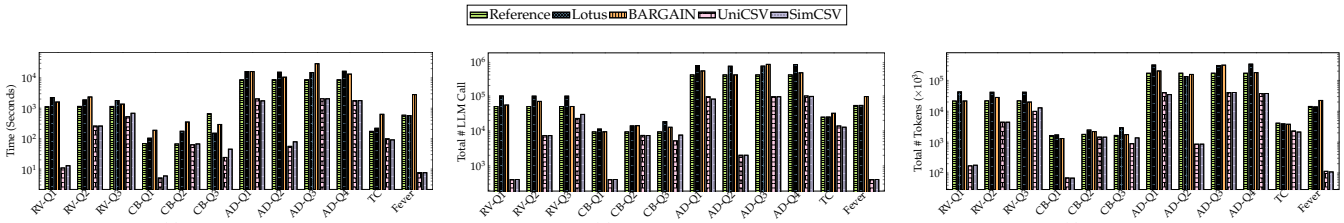
Efficiency. Fig. 4 illustrates LLM calls, execution time, and token consumption of different approaches. UniCSV and SimCSV achieve substantial efficiency gains by reducing LLM calls by 1.28-200× compared to Reference, 1.81-355× compared to Lotus, and 1.68-200× compared to BARGAIN. Since LLM calls dominate both runtime

Table 1: Profile of Test Semantic Queries

Dataset	Query	Query Type	Predicate	Ground-truth
IMDB-Review	RV-Q1	semantic filter	The {review} is positive.	available
	RV-Q2	semantic filter	The {review} explicitly recommends that others watch the movie.	GPT-4o
	RV-Q3	semantic filter	The {review} focuses on factual descriptions like technical details.	GPT-4o
Codebase	CB-Q1	semantic filter	The {AboutMe} contains a link to social media.	GPT-4o
	CB-Q2	semantic filter	The {AboutMe} text suggests that the user has either experience in computer science or expresses an interest in the field of computer science.	GPT-4o
	CB-Q3	semantic filter	The {AboutMe} focuses on factual identification details.	GPT-4o
TC	TC	semantic filter	The {comment} contains hate speech or offensive language.	available
Airdialogue	AD-Q1/Q2/Q3/Q4	semantic filter	The {dialogue} has outcome book/cancel/no_flight/no_reservation.	available
Fever	Fever	multi-column filter	The {claim} is supported by the {evidence}.	available

Table 2: The Accuracy and F1 Score of Semantic Filter Queries

	Method	RV-Q1	RV-Q2	RV-Q3	CB-Q1	CB-Q2	CB-Q3	AD-Q1	AD-Q2	AD-Q3	AD-Q4	TC	Fever
Acc.	Reference	0.9480	0.8229	0.9097	0.9731	0.7782	0.7033	0.8972	0.9139	0.8112	0.8737	0.7203	0.8258
	Lotus	0.9480	0.8679	0.9948	0.9595	0.6853	0.6996	0.8938	0.9066	0.9192	0.8690	0.5196	0.7396
	BARGAIN	0.9186	0.7881	0.9948	0.9681	0.5308	0.6493	0.7423	0.8972	0.3885	0.9563	0.7872	0.5177
	UniCSV	0.9307	0.7723	0.9527	0.9673	0.7810	0.7090	0.8948	0.9193	0.8723	0.8975	0.7477	0.7526
	SimCSV	0.9305	0.7718	0.9398	0.9671	0.7813	0.7376	0.8968	0.9193	0.8722	0.8977	0.7799	0.7524
F1	Reference	0.9489	0.7633	0.0880	0.6344	0.7557	0.6374	0.9056	0.8165	0.7002	0.1841	0.8060	0.8947
	Lotus	0.9489	0.7113	0.0972	0.4947	0.6115	0.3107	0.9029	0.8048	0.8393	0.1788	0.6007	0.7955
	BARGAIN	0.9140	0.5616	0.0296	0.5954	0.1979	0.0947	0.6878	0.7722	0.3429	0.3697	0.8589	0.5831
	UniCSV	0.9336	0.7097	0.0880	0.0623	0.7601	0.6340	0.9039	0.8249	0.7800	0.2179	0.8348	0.8588
	SimCSV	0.9335	0.7092	0.0906	0.0679	0.7555	0.6589	0.9052	0.8249	0.7799	0.2175	0.8550	0.8586



(a) Query Time (b) Total # LLM Calls (c) Total # Token

Figure 4: Comparisons of Query Time, # LLM Calls and Token Usage for Semantic Filter

Table 3: Hyper-parameter Effects on the Performance of UniCSV and SimCSV

Parameter	Value	RV-Q1 Acc.	RV-Q1 F1	CB-Q2 Acc.	CB-Q2 F1	AD-Q2 Acc.	AD-Q2 F1	RV Acc.	RV F1	CB-Q2 Acc.	CB-Q2 F1	AD-Q2 Acc.	AD-Q2 F1
		UniCSV						SimCSV					
# clusters	2	0.9079	0.9138	0.7780	0.7492	0.9168	0.8210	0.9082	0.9140	0.7739	0.7450	0.9168	0.8209
	4	0.9307	0.9336	0.7852	0.7601	0.9192	0.8248	0.9305	0.9335	0.7803	0.7539	0.9193	0.8249
	8	0.9367	0.9367	0.7829	0.7597	0.9202	0.8264	0.9388	0.9384	0.7774	0.7534	0.9202	0.8264
	16	0.9424	0.9438	0.7815	0.7564	0.9200	0.8262	0.9379	0.9396	0.7788	0.7546	0.9199	0.8260
sampling rate ξ (‰)	5	0.9307	0.9336	0.7852	0.7601	0.9192	0.8248	0.9305	0.9335	0.7803	0.7539	0.9193	0.8249
	10	0.9304	0.9334	0.7794	0.7518	0.9192	0.8248	0.9459	0.9470	0.7756	0.7493	0.9193	0.8249
	15	0.9304	0.9334	0.7802	0.7537	0.9193	0.8250	0.9439	0.9453	0.7828	0.7568	0.9192	0.8248
	20	0.9308	0.9337	0.7803	0.7536	0.9191	0.8246	0.9306	0.9335	0.7827	0.7565	0.9191	0.8248
	25	0.9307	0.9336	0.7788	0.7513	0.9192	0.8248	0.9307	0.9337	0.7861	0.7603	0.9191	0.8246
lower bound	0.10	0.9302	0.9333	0.7803	0.7547	0.9193	0.8249	0.9312	0.9341	0.7814	0.7551	0.9193	0.8249
	0.15	0.9307	0.9336	0.7852	0.7601	0.9192	0.8248	0.9305	0.9335	0.7803	0.7539	0.9193	0.8249
	0.20	0.9304	0.9334	0.7852	0.7595	0.9193	0.8249	0.9304	0.9334	0.7839	0.7575	0.9193	0.8248
	0.50	0.9304	0.9334	0.7959	0.7956	0.9193	0.8249	0.9304	0.9334	0.7953	0.7950	0.9193	0.8345

and token usage, these reductions translate directly into one to three orders of magnitude improvements across all datasets. For instance, on RV-Q1, UniCSV and SimCSV require only 404 calls, completing in under 13 seconds with approximately 170k tokens. In contrast, the baselines require tens of thousands of calls, resulting in runtimes exceeding 1,000 seconds and token usage exceeding 20 million. The higher cost incurred by Lotus and BARGAIN stems from their two-stage model-cascaded filtering. In many cases, the smaller 3B model fails to learn a threshold with sufficient precision, leading to more than half of the data being routed again to the larger 8B model. In the worst cases, i.e., RV-Q1 for Lotus and AD-Q3 for BARGAIN, the smaller model fails to filter any tuple, thereby increasing cost across all three metrics.

Effectiveness. Table 2 reports the Accuracy and F1 scores of all approaches across the 12 semantic filter queries. Overall, UniCSV and SimCSV demonstrate comparable performance with Reference, and outperform Lotus and BARGAIN, except CB-Q1. In contrast, BARGAIN exhibits highly unstable quality compared to Reference: while it achieves strong F1 scores and Accuracy on AD-Q4, it suffers from substantial degradation on CB-Q2 and AD-Q3. We speculate that this instability is because it uses the logits of LLM as the confidence signals of the predication, which is unreliable due to the inherent overconfidence of neural networks [15, 16]. One notable exception is the lower F1 score of our approaches on CB-Q1. This query explicitly mentions the entity 'social link', thereby possesses an extremely low selectivity of 0.033. The rare positives make Recall unstable with sampling failures under a default setting of $lb = 0.15$ (many samples contain few or no positives) and subsequent voting overconfidence bypasses the re-clustering. When we change lb to 0.01, the F1 score reaches 0.5825, but around half of the tuples are processed by linear LLM invocation.

The distinction between UniCSV and SimCSV is marginal in terms of Accuracy and F1 scores on well-clustered datasets, indicating their comparable effectiveness. In less clustered cases like CB-Q2 and AD-Q1, SimCSV exhibits a slight edge, showcasing more resilience to noisy or ambiguous cluster boundaries. This robustness enhances filter efficiency under challenging conditions. Moreover, UniCSV and SimCSV exhibit strong stability, consistently decreasing time and token consumption while maintaining effectiveness across datasets, queries, and varying sizes, from thousands (e.g., CB) to hundreds of thousands (e.g., AD).

4.3 Hyper-parameters Analysis

We analyze three hyper-parameters, number of clusters, sample ratio, and lower bound, that affect the effectiveness and efficiency of UniCSV and SimCSV as follows.

(1) Number of Clusters. As shown in Table 3, enlarging the cluster size enhances practical performance, although it does not impact the theoretical error bound. Accuracy and F1 score notably improve when #clusters range from 2 to 4, with gains diminishing thereafter.

For efficiency, depicted in Fig. 5, the number of clusters does not affect the total LLM calls, such as on RV-Q1 and AD-Q2, as the sample ratio dictates queries per cluster, maintaining a stable sampling volume. In smaller datasets like CB-Q2, the number of samples per cluster is primarily regulated by *min_sample*, resulting in a linear increase in total LLM calls with more clusters. Furthermore, in AD-Q2, setting the cluster count to 2 triggers re-clustering due to insufficient semantic distinction, leading to increased LLM calls, runtime, and token usage.

(2) Sample Ratio. As indicated in Table 3, the sample ratio has almost no impact on the Accuracy and F1 score of UniCSV and SimCSV, suggesting that a small sampling ratio suffices. We will delve into this further in the subsequent experiment in §4.5.

In Fig. 5, an increase in the sample ratio generally leads to a linear growth in LLM calls across most datasets. While on CB-Q2, LLM calls remain consistent despite the increase in sample ratio. This stability stems from the relatively small size of the dataset, where the sampling number is *min_sample* in most clusters. Notably, on RV-Q1 with SimCSV, a significant rise in LLM calls occurs when

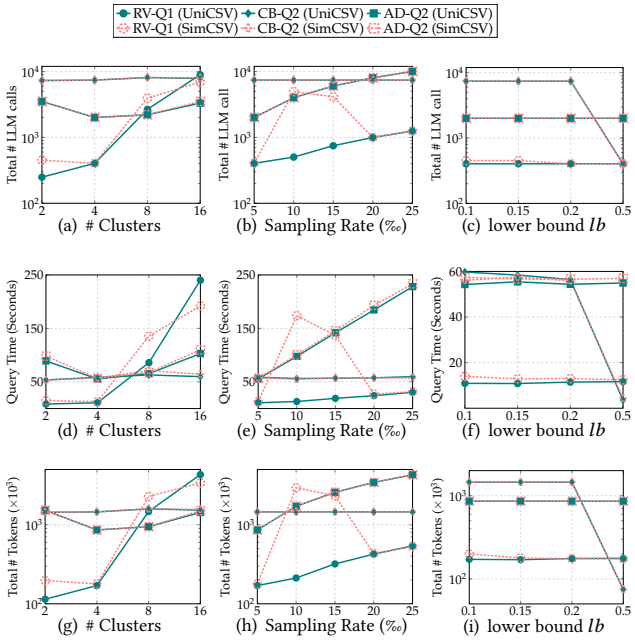


Figure 5: Hyper-parameter Effects on Query Time, # LLM Calls and Token Usage for Semantic Filter

the sampling rate reaches 10‰. This surge is attributed to partial fallbacks to linear processing, which occurs even within individual clusters. Unlike UniCSV, SimCSV employs a more adaptable voting strategy that may trigger more LLM queries when consensus is weak, even within internally coherent clusters. This illustrates the sensitivity of SimCSV to intra-cluster disagreement, particularly when the sampling density exceeds a threshold that reveals ambiguous cases.

(3) Lower Bound. By default, we set $ub = 1 - lb$. The number of LLM calls, Accuracy, and F1 score remain steady when lb is no greater than 0.2. Once lb hits 0.5 - enforcing voting irrespective of confidence - the LLM call count dramatically decreases, e.g., on CB-Q2, as this lb value averts linear processing and utilizes clustering. Furthermore, on the CB-Q2 query, setting lb to 0.5, which mandates voting solely based on consensus regardless of confidence, yields final accuracy and F1 scores surpassing those of the baselines. This implies that in situations where direct LLM inference is unreliable, i.e., on CB-Q2 queries, leveraging cluster-based consensus can enhance filtering performance beyond what traditional methods can achieve. These results illustrate that Accuracy and F1 scores stay relatively constant across a broad range of values for these parameters, showcasing the algorithm's robustness to moderate fluctuations in sampling and voting thresholds.

4.4 Impact of Re-clustering

We assess the impact of the recursive re-clustering mechanism through an ablation study, comparing both effectiveness metrics (Accuracy and F1) and efficiency metrics (# LLM calls and re-clustering time) with and without re-clustering. We use the default $lb = 0.15$ as re-clustering is enabled. When re-clustering is

Table 4: Ablation Study of Re-clustering

	Method	RV-Q1	RV-Q3	CB-Q1	CB-Q2	CB-Q3	AD-Q2
Acc.	UniCSV (w/o RC)	0.9229	0.9939	0.9677	0.7475	0.6403	0.9243
	SimCSV (w/o RC)	0.9230	0.9939	0.9677	0.7469	0.6403	0.9240
	UniCSV (w/ RC)	0.9301	0.9527	0.9673	0.7813	0.7090	0.9193
	SimCSV (w/ RC)	0.9305	0.9398	0.9671	0.7813	0.7376	0.9193
F1	UniCSV (w/o RC)	0.9272	0	0.0619	0.7010	0.5302	0.8347
	SimCSV (w/o RC)	0.9273	0	0.0619	0.7003	0.5302	0.8345
	UniCSV (w/ RC)	0.9336	0.0880	0.0623	0.7601	0.6340	0.8249
	SimCSV (w/ RC)	0.9335	0.0906	0.0679	0.7555	0.6589	0.8249
# Calls	UniCSV (w/o RC)	404	404	404	404	404	2008
	SimCSV (w/o RC)	404	404	404	404	404	2008
	UniCSV (w/ RC)	404	22760	404	7423	5196	2008
	SimCSV (w/ RC)	404	29683	404	7378	7586	2008
RC Time (%)	UniCSV (w RC)	0	0.44	0	1.47	3.27	0
	SimCSV (w RC)	0	0.29	0	1.37	2.05	0

Table 5: Gap between Theory and Practice

ϵ	UniCSV			SimCSV		
	Est.	**Err.**	ξ(‰)	**Est.**	**Err.**	ξ(‰)
0.10	0.9991	0.0628	13.0	0.9980	0.0622	26.4
0.15	0.9997	0.0633	6.4	0.9997	0.0635	12.7
0.20	0.9997	0.0636	3.9	0.9997	0.0637	7.8
0.25	0.9999	0.0637	2.7	0.9999	0.0635	5.4
0.30	0.9999	0.0635	2.0	0.9999	0.0637	4.0

disabled, we fix $lb = ub = 0.5$. This configuration forces the algorithm to commit to a voting decision regardless of confidence, bypassing the recursive re-clustering mechanism. The results are summarized in Table 4. We observe two distinct behavioral patterns across the evaluated queries. For queries RV-Q1, CB-Q1, AD-Q2, re-clustering is not triggered, yielding nearly identical performance between the two configurations. In RV-Q1 and AD-Q2, the initial clustering produces sufficiently pure clusters where the voting confidence consistently exceeds the threshold bounds, resulting in zero overhead in both additional LLM calls and runtime. In CB-Q1, the sampling failure from the low selectivity leads to over-confident voting results, and the re-clustering is not triggered. When using $lb = 0.01$ for CB-Q1, re-clustering will be triggered and the accuracy is improved. In contrast, queries RV-Q3, CB-Q2, and CB-Q3 exhibit substantial degradation when re-clustering is disabled. Accuracy drops by up to 9.7% and F1 scores decline by more than 12% in CB-Q3. Furthermore, the number of LLM calls increases significantly when re-clustering is enabled, indicating that a substantial portion of tuples in these cases require finer-grained cluster refinement. This underscores the mechanism's critical role in correcting low-confidence assignments and sustaining prediction quality for queries where the initial clustering fails to separate semantic categories cleanly. Importantly, despite the increase in LLM calls, the computational overhead of re-clustering remains modest. Even in the most demanding scenarios, re-clustering accounts for less than 3.3% of total runtime (CB-Q3), demonstrating that the iterative refinement process adds negligible latency relative to the dominant cost of LLM inference.

4.5 Theoretical Analysis

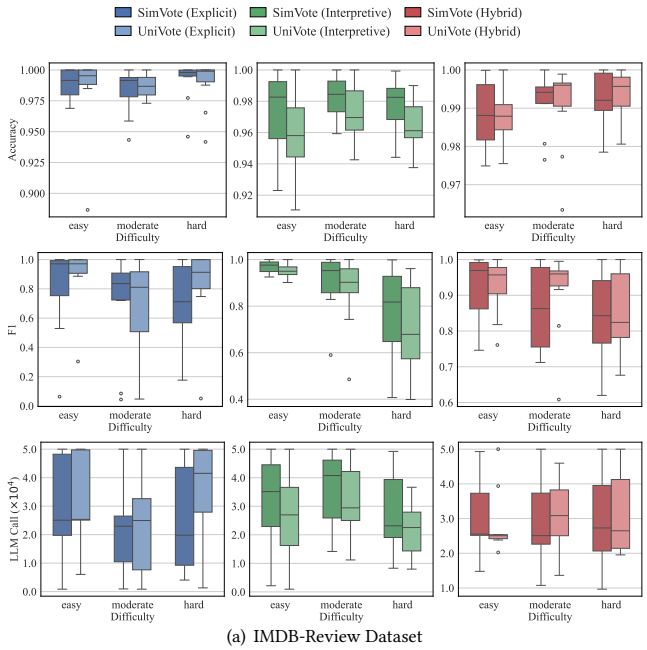
To investigate the gap between theory and practice, we analyze a representative cluster on RV, which contains 14,608 embeddings, and the confidence interval within this cluster is measured at 0.9942, using voting algorithms UniCSV and SimCSV. Specifically, we vary the theoretical error bound ϵ to compute the required sample ratio ξ based on Theorems 3.3 and 3.6, then we report the empirical voting output, denoted as Est., and the error between LLM-inferred output and voting output, denoted as Err., i.e., Est. is the ratio of sampled x_i for which $f(t_i) = x_i$, and Err. is the difference between the sampled instances and all instances. We set $lb = 0.15$ and $ub = 1 - lb$ by default, and set $l = 0.9996$, i.e., the failure probability is less than 0.579%.

The results are shown in Table 5, for both UniVote and SimVote, the practical error is much smaller than the theoretical guaranteed error, and increasing the theoretical error bound ϵ leads to a substantial reduction in the required sample ratio. Therefore, this supports that small sample ratios are adequate to guarantee strong theoretical error bounds and maintain high empirical effectiveness. Moreover, although SimVote theoretically yields looser error bounds, its sampling rate ξ is always larger than UniVote. Nevertheless, the empirical performance of SimVote remains comparable to UniVote, suggesting that its semantic voting mechanism is robust even under relaxed theoretical constraints.

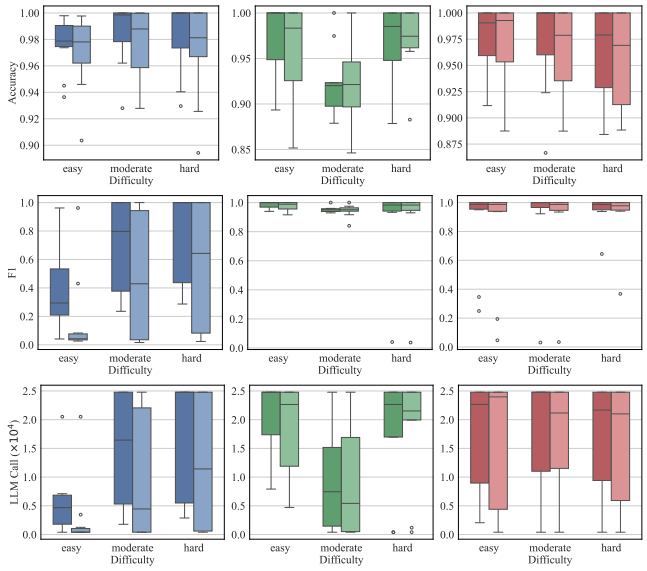
4.6 Testing on Synthetic Queries

To evaluate CSV over a broader and more diverse predicate space, we construct a synthetic query suite with controlled variations in semantics and difficulty. Concretely, we instruct GPT 5.2 to generate three types of queries for two datasets (IMDB-Review and TC): (1) queries that explicitly refer entities/concepts in the tuple texts (*Explicit* predicates); (2) queries that convey abstract, subjective, interpretive semantics (*Interpretive* predicates); and (3) queries that combine the explicit mentions with interpretive semantics (*Hybrid* predicates). For each type, we generate three difficulty levels (easy/moderate/hard), where difficulty reflects semantic subtlety and linguistic complexity. Each (type, difficulty) group contains 10 distinct queries, yielding 90 queries per dataset and a total of 180 queries across the two datasets. We generate these queries by providing sampled tuples and few-shot demonstrations to GPT-5.2. We evaluate UniCSV, SimCSV, and linear LLM invocation baseline (Reference) on these synthetic queries, respectively. For predicates with strong lexical anchoring (Explicit and Hybrid), we use a hybrid clustering distance that combines lexical and semantic signals (weighted sum of BM25 similarity and Euclidean distance), while for Interpretive predicates, we keep the vanilla Euclidean distance. We use the results of Reference as the proxy of ground-truth.

Fig. 6 reports the box-plot over Accuracy, F1, and # LLM calls across datasets, query types, and difficulty levels. Overall, UniCSV and SimCSV achieve high Accuracy across the synthetic suite, while consistently reducing LLM calls relative to the baseline Reference. We also observe nuanced failure modes that help characterize the generality of our method. For hard Interpretive/Hybrid predicates, F1 drops relatively to easy/moderate predicates, this is preliminarily caused by extremely low selectivity as there are very few true positives, making the random samples hardly contain enough positives for reliable voting. In such cases, Accuracy can remain high due to label skew (true negatives), while F1 reveals missed rare positives. Similar phenomena also occur in Explicit predicates, where CSV requires more LLM calls to achieve comparable F1 scores. The hybrid of BM25 and embedding distance improves clustering quality for



(a) IMDB-Review Dataset



(b) Comment Dataset

Figure 6: UniCSV and SimCSV on Synthetic Queries

these predicates. In summary, these results verify that CSV supports a broader predicate space, and reveal a clear limitation regime of our method.

5 RELATED WORK

General ML-based Analytical Query Processing. With the success of deep learning, Video database management systems (VDBMS) [5, 18, 19, 21, 37, 48] are designed and developed for video data analysis, where deep learning models are used as UDFs in DBMS for image classification, object detection, etc. Oriented for video analytical queries, these systems involve a wide range of optimization techniques, encompassing end-to-end optimization of the preprocessing and query pipeline [21], declarative query hints [37], fine-grained model selection [8], query-agnostic semantic index [5], and fast evaluation algorithms for specific operators [5, 18]. Apart from video analysis, probabilistic predicates [13, 30, 49] with ML models are used in a broader range of applications, where cheaper proxy models are constructed and their correlations are exploited. For hybrid queries of ML and relational operators, RAVEN [33] conducts cross optimization of ML and relational queries, exploiting data statistics to select across multiple executions. In addition, some systems also adopt adaptive query processing, which adjusts query plans and resource allocation, or choose cheaper proxy models, following runtime statistics dynamically. ABAE [20], InQuest [38] focus on optimizing aggregate queries with ML models by introducing sampling-based aggregate algorithms using proxy models.

LLM-based Semantic Query Processing. LLMs open up a new query paradigm, semantic query processing, for structured data [14, 27] and unstructured data [4, 40, 44]. DocETL [40], ZenDB [25], DocWanger [41], Aryn [4] and Unify [44] leverage LLMs for analytics of unstructured text by semantic queries, where the query languages range from extended SQL dialect [25], declarative operations [40] to natural language [4, 41, 44]. In these document analytics systems, LLMs are also used as optimization agents in the procedure of query evaluation, such as query planning [4], query rewriting, prompt refinement and operation decomposition [41]. UQE [11] extends SQL by allowing natural language join and selection predicates, which intrinsically support semantic join and semantic filter by LLMs. Palimpzest [26] is an LLM-powered analytical system, with logical and physical optimizations to reduce the query time and financial cost of LLM invocations. Abacus [39] is an optimizer built on Palimpzest, which integrates a Pareto optimization algorithm and cost estimation for query planning. Lotus [34] formulates LLM-based semantic operations in tables. Furthermore, Lotus proposes optimized algorithms for expensive semantic operations, with batched inference and LLM cascading. BARGAIN [50] utilizes the same model cascade framework as Lotus. Beyond accuracy alone, it ensures provable bounds on recall, precision, and accuracy for semantic operations. [3] evaluates the semantic query performance of PostgreSQL with pgAI and DuckDB with FlockMTL [14], highlighting the challenges of enforcing structured outputs, batch processing, and query planning. In these systems, semantic filters are fundamental operators.

6 CONCLUSION

This paper presents an efficient and scalable approach, Clustering-Sampling-Voting (CSV), to semantic filtering, addressing the core computational challenges of integrating LLMs into data systems. By clustering similar tuples, sampling representatives, and inferring labels through theoretical guarantees voting, CSV reduces redundant LLM calls while maintaining comparable effectiveness. Extensive experiments validate the efficacy of our proposed CSV, showcasing a reduction in the number of LLM calls to 1.28-200× compared to Reference, and 1.81-355× compared to Lotus.

REFERENCES

[1] [n. d.]. The technical report of CSV. https://github.com/Anto-an/CSV_SemanticFilter/blob/main/TR.pdf.

[2] 2023. LiteLLM: Unified API for LLMs. https://github.com/BerriAI/litellm.

[3] Kerem Akillioglu, Anurag Chakraborty, Sairaj Voruganti, and M. Tamer Özsu. 2025. Research Challenges in Relational Database Management Systems for LLM Queries. arXiv:2508.20912 [cs.DB]

[4] Eric Anderson, Jonathan Fritz, Austin Lee, Bohou Li, Mark Lindblad, Henry Lindeman, Alex Meyer, Parth Parmar, Tanvi Ranade, Mehul A. Shah, Benjamin Sowell, Dan Tecuci, Vinayak Thapliyal, and Matt Welsh. 2024. The Design of an LLM-powered Unstructured Analytics System. *CoRR* abs/2409.00847 (2024). arXiv:2409.00847

[5] Jaeho Bang, Gaurav Tarlok Kakkar, Pramod Chunduri, Subrata Mitra, and Joy Arulraj. 2023. SEIDEN: Revisiting Query Processing in Video Database Systems. *Proc. VLDB Endow.* 16, 9 (2023), 2289–2301.

[6] Rémi Bardenet and Odalric-Ambrym Maillard. 2015. Concentration inequalities for sampling without replacement. (2015).

[7] Asim Biswal, Liana Patel, Siddarth Jha, Amog Kamsetty, Shu Liu, Joseph E Gonzalez, Carlos Guestrin, and Matei Zaharia. 2024. Text2sql is not enough: Unifying ai and databases with tag. *arXiv preprint arXiv:2408.14717* (2024).

[8] Jiashen Cao, Karan Sarkar, Ramyad Hadidi, Joy Arulraj, and Hyesoon Kim. 2022. FiGO: Fine-Grained Query Optimization in Video Analytics. In *SIGMOD*. ACM, 559–572.

[9] Ilias Chalkidis, Abhik Jana, Dirk Hartung, Michael J. Bommarito II, Ion Androutsopoulos, Daniel Martin Katz, and Nikolaos Aletras. 2022. LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. In *Proc. ACL 2022*. Association for Computational Linguistics, 4310–4330.

[10] Haoang Chi, He Li, Wenjing Yang, Feng Liu, Long Lan, Xiaoguang Ren, Tongliang Liu, and Bo Han. 2024. Unveiling causal reasoning in large language models: Reality or mirage? *Advances in Neural Information Processing Systems* 37 (2024), 96640–96670.

[11] Hanjun Dai, Bethany Wang, Xingchen Wan, Bo Dai, Sherry Yang, Azade Nova, Pengcheng Yin, Mangpo Phothilimthana, Charles Sutton, and Dale Schuurmans. 2024. UQE: A Query Engine for Unstructured Databases. *Advances in Neural Information Processing Systems* 37 (2024), 29807–29838.

[12] Thomas Davidson, Dana Warmsley, Michael W. Macy, and Ingmar Weber. 2017. Automated Hate Speech Detection and the Problem of Offensive Language. In *ICWSM*. AAAI Press, 512–515.

[13] Dujian Ding, Sihem Amer-Yahia, and Laks V. S. Lakshmanan. 2022. On Efficient Approximate Queries over Machine Learning Models. *Proc. VLDB Endow.* 16, 4 (2022), 918–931.

[14] Anas Dorbani, Sunny Yasser, Jimmy Lin, and Amine Mhedhbi. 2025. Beyond Quacking: Deep Integration of Language Models and RAG into DuckDB. *Proc. VLDB Endow.* 18, 12 (2025), 5415–5418.

[15] Yarin Gal and Zoubin Ghahramani. 2016. Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. In *Proc. ICML 2016*, Vol. 48. JMLR.org, 1050–1059.

[16] Jiahui Geng, Fengyu Cai, Yuxia Wang, Heinz Koeppl, Preslav Nakov, and Iryna Gurevych. 2024. A Survey of Confidence Estimation and Calibration in Large Language Models. In *Proc. NAACL 2024*. Association for Computational Linguistics, 6577–6595. doi:10.18653/V1/2024.NAACL-LONG.366

[17] Karen Sparck Jones, Steve Walker, and Stephen E. Robertson. 2000. A probabilistic model of information retrieval: development and comparative experiments - Part 1. *Inf. Process. Manag.* 36, 6 (2000), 779–808.

[18] Daniel Kang, Peter Bailis, and Matei Zaharia. 2019. BlazeIt: Optimizing Declarative Aggregation and Limit Queries for Neural Network-Based Video Analytics. *Proc. VLDB Endow.* 13, 4 (2019), 533–546.

[19] Daniel Kang, Edward Gan, Peter Bailis, Tatsunori Hashimoto, and Matei Zaharia. 2020. Approximate Selection with Guarantees using Proxies. *Proc. VLDB Endow.* 13, 11 (2020), 1990–2003.

[20] Daniel Kang, John Guibas, Peter Bailis, Tatsunori Hashimoto, Yi Sun, and Matei Zaharia. 2021. Accelerating Approximate Aggregation Queries with Expensive Predicates. *Proc. VLDB Endow.* 14, 11 (2021), 2341–2354.

[21] Daniel Kang, Ankit Mathur, Teja Veeramacheneni, Peter Bailis, and Matei Zaharia. 2020. Jointly Optimizing Preprocessing and Inference for DNN-based Visual Analytics. *Proc. VLDB Endow.* 14, 2 (2020), 87–100.

[22] Daniel Kang, Francisco Romero, Peter D. Bailis, Christos Kozyrakis, and Matei Zaharia. 2022. VIVA: An End-to-End System for Interactive Video Analytics. In *CIDR*. www.cidrdb.org.

[23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.

[24] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2024).

[25] Yiming Lin, Madelon Hulsebos, Ruiying Ma, Shreya Shankar, Sepanta Zeighami, Aditya G. Parameswaran, and Eugene Wu. 2024. Towards Accurate and Efficient Document Analytics with Large Language Models. *CoRR* abs/2405.04674 (2024). arXiv:2405.04674

[26] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, et al. 2025. Palimpzest: Optimizing ai-powered analytics with declarative query processing. In *CIDR*. 2.

[27] Shu Liu, Asim Biswal, Amog Kamsetty, Audrey Cheng, Luis Gaspar Schroeder, Liana Patel, Shiyi Cao, Xiangxi Mo, Ion Stoica, Joseph E. Gonzalez, and Matei Zaharia. 2025. Optimizing LLM Queries in Relational Data Analytics Workloads. In *Eighth Conference on Machine Learning and Systems*.

[28] Shicheng Liu, Jialiang Xu, Wesley Tjangnaka, Sina J. Semnani, Chen Jie Yu, and Monica Lam. 2024. SUQL: Conversational Search over Structured and Unstructured Data with Large Language Models. In *NAACL*. Association for Computational Linguistics, 4535–4555.

[29] Xutong Liu, Baran Atalar, Xiangxiang Dai, Jinhang Zuo, Siwei Wang, John Lui, Wei Chen, and Carlee Joe-Wong. 2025. Semantic Caching for Low-Cost LLM Serving: From Offline Learning to Online Adaptation. *arXiv preprint arXiv:2508.07675* (2025).

[30] Yao Lu, Aakanksha Chowdhery, Srikanth Kandula, and Surajit Chaudhuri. 2018. Accelerating Machine Learning Inference with Probabilistic Predicates. In *SIGMOD*. ACM, 1493–1508.

[31] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. 2011. Learning Word Vectors for Sentiment Analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*. 142–150.

[32] Ramaswami Mohandoss. 2024. Context-based semantic caching for llm applications. In *2024 IEEE Conference on Artificial Intelligence*. IEEE, 371–376.

[33] Kwanghyun Park, Karla Saur, Dalitso Banda, Rathijit Sen, Matteo Interlandi, and Konstantinos Karanasos. 2022. End-to-end Optimization of Machine Learning Prediction Queries. In *SIGMOD*. ACM, 587–601.

[34] Liana Patel, Siddharth Jha, Melissa Z. Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Towards AI-Based Data Analytics with Accuracy Guarantees. *Proc. VLDB Endow.* 18, 11 (2025), 4171–4184.

[35] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake VanderPlas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *J. Mach. Learn. Res.* 12 (2011), 2825–2830.

[36] Sajal Regmi and Chetan Phakami Pun. 2024. Gpt semantic cache: Reducing llm costs and latency via semantic embedding caching. *arXiv preprint arXiv:2411.05276* (2024).

[37] Francisco Romero, Johann Hauswald, Aditi Partap, Daniel Kang, Matei Zaharia, and Christos Kozyrakis. 2022. Optimizing Video Analytics with Declarative Model Relationships. *Proc. VLDB Endow.* 16, 3 (2022), 447–460.

[38] Matthew Russo, Tatsunori Hashimoto, Daniel Kang, Yi Sun, and Matei Zaharia. 2023. Accelerating Aggregation Queries on Unstructured Streams of Data. *Proc. VLDB Endow.* 16, 11 (2023), 2897–2910.

[39] Matthew Russo, Sivaprasad Sudhir, Gerardo Vitagliano, Chunwei Liu, Tim Kraska, Samuel Madden, and Michael Cafarella. 2025. Abacus: A Cost-Based Optimizer for Semantic Operator Systems. *arXiv preprint arXiv:2505.14661* (2025).

[40] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *Proc. VLDB Endow.* 18, 9 (2025), 3035–3048.

[41] Shreya Shankar, Bhavya Chopra, Mawil Hasan, Stephen Lee, Björn Hartmann, Joseph M. Hellerstein, Aditya G. Parameswaran, and Eugene Wu. 2025. Steering Semantic Data Processing With DocWrangler. *CoRR* abs/2504.14764 (2025). arXiv:2504.14764

[42] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: a Large-scale Dataset for Fact Extraction and VERification. In *NAACL-HLT*. Association for Computational Linguistics, 809–819.

[43] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[44] Jiayi Wang and Jianhua Feng. 2025. Unify: An Unstructured Data Analytics System. In *ICDE*. IEEE, 4662–4674.

[45] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text Embeddings by Weakly-Supervised Contrastive Pre-training. *arXiv preprint arXiv:2212.03533* (2022).

[46] Wei Wei, Quoc Le, Andrew Dai, and Jia Li. 2018. AirDialogue: An Environment for Goal-Oriented Dialogue Research. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 3844–3854.

[47] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv:2309.07597 [cs.CL]

[48] Ziyang Xiao, Dongxiang Zhang, Zepeng Li, Sai Wu, Kian-Lee Tan, and Gang Chen. 2023. DoveDB: A Declarative and Low-Latency Video Database. *Proc. VLDB Endow.* 16, 12 (2023), 3906–3909.

[49] Zhihui Yang, Zuozhi Wang, Yicong Huang, Yao Lu, Chen Li, and X. Sean Wang. 2022. Optimizing Machine Learning Inference Queries with Correlative Proxy Models. *Proc. VLDB Endow.* 15, 10 (2022), 2032–2044.

[50] Sepanta Zeighami, Shreya Shankar, and Aditya G. Parameswaran. 2025. Cut Costs, Not Accuracy: LLM-Powered Data Processing with Guarantees. *Proc. ACM Manag. Data* 3, 6 (2025), 1–26. doi:10.1145/3769776

[51] Jingqing Zhang, Yao Zhao, Mohammad Saleh, and Peter Liu. 2020. Pegasus: Pre-training with extracted gap-sentences for abstractive summarization. In *International conference on machine learning*. PMLR, 11328–11339.

[52] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176* (2025).

A PROOF OF COROLLARY 3.4

Corollary A.1. *Let $X = \{x_1, ..., x_n\}$ be a finite population of size n, where $x_1, \cdots, x_n$ are binary variables with a mean μ. Let $c_1, ..., c_k$ be sampled without replacement from X, and each c_i is associated with a weight w_i, where $w_i \geq 0$ and $\sum_{i=1}^{k} w_i = 1$. Let $\hat{\mu}$ be the mean of $c_1, \cdots, c_k$, i.e., $\hat{\mu} = \frac{1}{k}\sum_{i=1}^{k} w_i c_i$. For any $\epsilon > 0$, we have*

$$Pr[|\hat{\mu}_w - \mu| \geq \epsilon] \leq 2\exp\left(\frac{-3k\epsilon^2}{(6\hat{\sigma}^2 + 2\epsilon)v} \cdot \frac{n-k}{n-1}\right),$$

where $\hat{\sigma}^2 = \frac{1}{k-1}\sum_{i=1}^{k}(c_i - \hat{\mu})^2$ is the sample standard deviation, and v is a constant such that $\max_i w_i \leq \frac{v}{k}$.

Proof Sketch: $\hat{\mu}_w - \mu = \sum_{i=1}^{k} w_i c_i - \sum_{i=1}^{k} w_i \mu = \sum_{i=1}^{k} w_i (c_i - \mu)$. Let $y_i = w_i(c_i - \mu)$, we have $\hat{\mu}_w - \mu = \sum_{i=1}^{k} y_i$. We have the mean of y_i equals to 0 by the law of big numbers, since c_i is sampled from X with mean μ. Apply Lemma 3.2 for y_i, we have

$$Pr\left[\left|\frac{1}{k}\sum_{i=1}^{k} y_i\right| \geq \epsilon\right] \leq 2\exp\left(-\frac{k\epsilon^2}{2\hat{\sigma}_y^2 + 2R\epsilon/3} \cdot \frac{n-k}{n-1}\right). \quad (6)$$

As c_i i.i.d. of x_i, $\hat{\sigma}_y^2 = \frac{1}{k-1}\sum_{i=1}^{k}(w_i(c_i - \mu))^2 = \frac{1}{k}\hat{\sigma}^2 \sum_{i=1}^{k} w_i^2$. Since $|y_i| = w_i(c_i - \mu) \leq 1$, $R \leq \max_i w_i \leq \frac{v}{k}$, $Pr[|\hat{\mu}_w - \mu| \geq \epsilon] = Pr\left[\left|\sum_{i=1}^{k} y_i\right| \geq k\epsilon\right]$, and $\sum_{i=1}^{k} w_i^2 \leq v/k$, According to Eq. (6), we have

$$Pr[|\hat{\mu}_w - \mu| \geq \epsilon] = Pr\left[\left|\sum_{i=1}^{k} y_i\right| \geq \epsilon\right] = Pr\left[\left|\frac{1}{k}\sum_{i=1}^{k} y_i\right| \geq \frac{\epsilon}{k}\right]$$

$$\leq 2\exp\left(-\frac{k(\epsilon/k)^2}{2\hat{\sigma}_y^2 + 2\epsilon/3k} \cdot \frac{n-k}{n-1}\right) = 2\exp\left(\frac{-3k\epsilon^2}{(6\hat{\sigma}^2 + 2\epsilon)v} \cdot \frac{n-k}{n-1}\right).$$

□

□

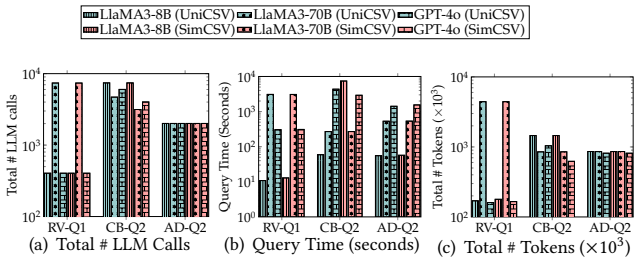
B IMPACT OF EMBEDDING MODELS & LLMS

(1) Impact of Different Embedding Models. We study how the embedding model affects UniCSV and SimCSV by comparing BGE-Large-en (BGE) [47], Qwen-0.6B (Qwen) [52], and the default E5-Large (E5)[45] model across all datasets, with results in Table 7 and Fig. 8. For effectiveness, gauged by Accuracy and F1, differences are relatively small. BGE is slightly better on RV-Q1 but performs slightly worse on AD-Q2. These results imply that embedding quality has a limited impact on the ultimate result quality. Efficiency metrics unveil more notable distinctions. BGE leads to increased resource utilization on both RV-Q1 and AD-Q2 due to its embeddings generating less coherent clusters, necessitating more LLM queries for resolution. Qwen performs suboptimally on RV-Q1. Nonetheless, all embedding models maintain significantly enhanced efficiency compared to the baselines, achieving speedups ranging from 1.1× to 18.95×, showcasing the robustness of the voting strategy even with suboptimal embedding models.

(2) Impact of Different Backbones. We evaluate the performance of three backbone LLMs: LlaMA3-8B, a quantized int4 version of LlaMA3-70B, and GPT-4o. The LlaMA models are deployed by local vLLM engine while GPT-4o is used by remote invocation. The results are detailed in Table 6 and Fig. 7. All three models exhibit comparable efficiency and effectiveness metrics, suggesting that UniCSV and SimCSV are not overly sensitive to model selection. LlaMA3-70B excels in performance on RV-Q1, indicating that its larger capacity enables finer semantic discrimination in intricate cases. GPT-4o shows a significant advantage on CB-Q2, surpassing other models by approximately 10% in Accuracy. This improvement likely stems from GPT-4o's superior handling of complex reasoning tasks. However, GPT-4o is not always dominant; since many queries are adequately resolved by smaller models, and its utilization can sometimes trigger content management limitations. For efficiency, LLM call volume and token usage remain relatively consistent across CB-Q2 and AD-Q2. On RV-Q1, LlaMA3-70B incurs a higher number of calls, reflecting its sensitivity to nuanced semantic boundaries. Nonetheless, it still achieves a 6.76× speedup over baseline methods. Query latency varies across models due to variations in size, server throughput, and access constraints.

Table 6: Semantic Filter with Different LLM Backbones

Algo.	Model	RV-Q1		CB-Q2		AD-Q2	
		Acc.	F1	Acc.	F1	Acc.	F1
UniCSV	LlaMa-8B	0.9307	0.9336	0.7799	0.7517	0.9190	0.8248
	LlaMa-70B	0.9520	0.9523	0.7715	0.8079	0.9194	0.8251
	GPT-4o	0.9306	0.9335	0.8870	0.8814	0.9194	0.8250
SimCSV	LlaMa-8B	0.9305	0.9335	0.7803	0.6988	0.9193	0.8249
	LlaMa-70B	0.9521	0.9524	0.7701	0.8068	0.9194	0.8251
	GPT-4o	0.9305	0.9335	0.8666	0.8687	0.9193	0.8250

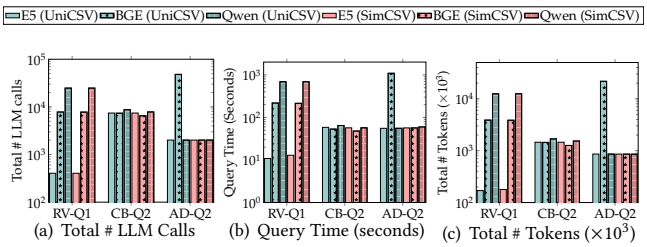


(a) Total # LLM Calls (b) Query Time (seconds) (c) Total # Tokens ($\times 10^3$)

Figure 7: Comparisons on Different LLM Backbones

Table 7: Semantic Filter with Different Embedding Models

Algo.	Model	RV-Q1		CB-Q2		AD-Q2	
		Acc.	F1	Acc.	F1	Acc.	F1
UniCSV	E5	0.9307	0.9336	0.7852	0.7601	0.9192	0.8248
	BGE	0.9527	0.9531	0.7827	0.7594	0.9122	0.8036
	Qwen	0.9227	0.9235	0.7777	0.7540	0.9172	0.8192
SimCSV	E5	0.9305	0.9335	0.7815	0.7555	0.9193	0.8249
	BGE	0.9520	0.9524	0.7806	0.7587	0.8947	0.7755
	Qwen	0.9223	0.9231	0.7794	0.7500	0.9172	0.8192



(a) Total # LLM Calls (b) Query Time (seconds) (c) Total # Tokens ($\times 10^3$)

Figure 8: Comparisons of Using Different Embedding Models